Algorithm 1: Budget-Aware MCTS with Adaptive Action Sampling

Input: Time budget T, tree root x_0, progressive widening exponent α
Output: Best action a^*

```
% Hyperparameters: W_low = 15, W_high = 50, Nw = 100, Nr = 200

% Initialize with equal sampling (no bias at episode start)
 1:  p ← (p_agg, p_slow, p_feas) = (1/3, 1/3, 1/3)
 2:  t_start ← Clock();  t_deadline ← t_start + T;  n ← 0

── Phase 1: Warmup (measure iteration rate) ──────────
 3:  while n < Nw and Clock() < t_deadline do
 4:      GrowTree(x₀, p);  n ← n + 1
 5:  end while

── Phase 2: Estimate budget & adapt weights ──────────
 6:  t̄_iter ← (Clock() − t_start) / n                    ▷ avg time per iter
 7:  N_total ← n + ⌊(t_deadline − Clock()) / t̄_iter⌋      ▷ estimated total iters
 8:  Ŵ ← N_total^α                                       ▷ expected root width
 9:  p ← ComputeWeights(Ŵ)

── Phase 3: Adaptive search with re-calibration ──────
10:  while Clock() < t_deadline do
11:      GrowTree(x₀, p);  n ← n + 1
12:      if n mod Nr = 0 then                            ▷ re-calibrate
13:          t̄_iter ← (Clock() − t_start) / n
14:          N_total ← n + ⌊(t_deadline − Clock()) / t̄_iter⌋
15:          Ŵ ← N_total^α
16:          p ← ComputeWeights(Ŵ)
17:      end if
18:  end while

19:  return a* ← argmax over children(x₀) of visits(a)

function ComputeWeights(Ŵ):
20:  τ ← clamp((Ŵ − 15) / 35,  0,  1)                    ▷ budget interpolant

    Tight budget (τ → 0): favor braking for safety
    Ample budget (τ → 1): favor GP prior + aggressive for speed

21:  p_slow ← 0.70 − 0.60 · τ                            ▷ 0.70 → 0.10
22:  p_feas ← 0.20 + 0.35 · τ                            ▷ 0.20 → 0.55
23:  p_agg  ← 1 − p_slow − p_feas                        ▷ 0.10 → 0.35
24:  return (p_agg, p_slow, p_feas)
end function

function SampleAction(p):
                                  ▷ used inside GrowTree on curves
25:  Draw u ~ Uniform(0, 1)
26:  if u < p_agg then
27:      a_accel ~ Uniform(0, a_max)                     ▷ aggressive (accelerate)
28:  else if u < p_agg + p_slow then
29:      a_accel ~ Uniform(−a_max, 0)                    ▷ slow (brake)
30:  else
31:      a_accel ~ N(μ_GP, σ²_GP)                        ▷ feasible (GP prior)
32:  end if
33:  a_steer ~ N(μ_GP^sv, σ²_GP^sv)                      ▷ steering from GP prior
34:  return (a_accel, a_steer)
end function
```

Hyperparameters:

Symbol	Value	Description
T	35 ms	MCTS time budget per planning step
α	0.5	Action progressive widening exponent
W_{low}	15	Root width below which budget is "tight"
W_{high}	50	Root width above which budget is "ample"
N_w	100	Warmup iterations (equal 1/3 sampling)
N_r	200	Re-calibration interval

Endpoint sampling distributions:

Budget regime	p_{agg}	p_{slow}	p_{feas}	Behavior
Tight ($\hat{W} \leq 15$)	0.10	0.70	0.20	Mostly braking → safety
Ample ($\hat{W} \geq 50$)	0.35	0.10	0.55	GP + accel → speed
Episode start	1/3	1/3	1/3	Equal (before calibration)